

CMSC 471, PA1: A* Search with DOGCAT

Tejas Tyagi, Section 02

September 25, 2026

1 Heuristic design and admissibility (15 pts)

Let n be the current word and let $D(n)$ be the number of positions where n differs from the goal (the Hamming distance). Two facts hold for every cost measure. First, each action changes exactly one position, so one action can fix at most one wrong position. Second, for each wrong position i , the last action that writes into position i must put $\text{goal}[i]$ there, and these last writes are separate actions for separate positions.

1.1 Steps

Formula: $h_{\text{steps}}(n) = D(n)$.

Each incorrect position needs at least one change, and one change can only fix one position, so there must be at least $D(n)$ more moves. Every move costs 1 under the steps measure, so the remaining cost is at least $D(n)$. That means $D(n)$ can never be bigger than the real remaining cost, which is what admissibility needs. When the word already matches the goal, $D = 0$ and $h = 0$.

1.2 Scrabble

Formula:

$$h_{\text{scrabble}}(n) = \sum_{i: n[i] \neq \text{goal}[i]} \text{val}(\text{goal}[i]),$$

where $\text{val}(\ell)$ is the Scrabble value of letter ℓ (A,E,I,O,U,L,N,S,T,R= 1; D,G= 2; B,C,M,P= 3; F,H,V,W,Y= 4; K= 5; J,X= 6; Q,Z= 10).

Here a move costs the Scrabble value of the letter I put in. For every position that is still wrong, at some point I have to write the goal letter into it, and that write costs $\text{val}(\text{goal}[i])$. Those writes happen at different positions, so they are all separate moves. Adding them up gives $\sum_i \text{val}(\text{goal}[i])$, and any other moves I make on top of that only add more cost, so this sum can never be more than the real remaining cost. I use the goal letter $\text{goal}[i]$ and not the current letter $n[i]$ because the cost is charged on the letter I insert.

1.3 Frequency

Formula:

$$h_{\text{freq}}(n) = \begin{cases} 0 & \text{if } D(n) = 0, \\ D(n) + \text{rarity}(\text{goal}) & \text{if } D(n) \geq 1. \end{cases}$$

A move that lands on word w costs $1 + \text{rarity}(w)$, and rarity is never negative, so every move costs at least 1. If the word is not the goal yet, I still need at least $D(n)$ moves, and the very last move

has to land on the goal word, which costs exactly $1 + \text{rarity}(\text{goal})$. The other $D(n) - 1$ moves cost at least 1 each. Putting that together, the remaining cost is at least $(D(n) - 1) + (1 + \text{rarity}(\text{goal})) = D(n) + \text{rarity}(\text{goal})$.

2 Does the heuristic help? (10 pts)

I ran uniform-cost search and A* on both word pairs under all three cost measures, using a fresh DC instance for each run and reading the “paths expanded” count reported by the supplied search functions with `display=True`.

Word pair	Cost	UCS cost	UCS nodes	A* cost	A* nodes	Reduction
WHY-ASK	steps	5.00	773	5.00	16	97.9%
WHY-ASK	scrabble	9.00	765	9.00	13	98.3%
WHY-ASK	frequency	38.86	808	38.86	634	21.5%
BEAR-DUCK	steps	4.00	691	4.00	6	99.1%
BEAR-DUCK	scrabble	11.00	2981	11.00	6	99.8%
BEAR-DUCK	frequency	38.24	1084	38.24	302	72.1%

1. Did UCS and A* return the same optimal cost? Yes. Looking down the two cost columns in my table, every row matches: 5.00, 9.00, 38.86, 4.00, 11.00, 38.24. That is what I expected, since my heuristics are admissible and A* with an admissible heuristic still returns an optimal-cost path. In a few runs the actual word sequence A* returned was different from the one UCS returned, but the total cost was the same.

2. Largest reduction per word pair. For both word pairs the Scrabble row had the biggest cut in expanded nodes: 98.3% for WHY-ASK and 99.8% for BEAR-DUCK. The steps row was close behind (97.9% and 99.1%).

3. Least informative per word pair. For both pairs the frequency row helped the least: 21.5% for WHY-ASK and 72.1% for BEAR-DUCK. The BEAR-DUCK Scrabble row is a good example of how much the heuristic mattered: UCS expanded 2981 nodes there but A* only expanded 6.

4. Explanation using the heuristic definitions. The gap between the steps/Scrabble rows and the frequency rows comes straight from how much of the real cost each heuristic can account for. For steps every leftover edit is worth exactly 1, and for Scrabble my heuristic already adds the exact value of each goal letter that still has to be placed, so in both cases the estimate lands close to the true remaining cost and A* does not wander. Frequency is different. My frequency heuristic is $D + \text{rarity}(\text{goal})$, so it only counts the goal word’s rarity, but the real cost also has a $1 + \text{rarity}(w)$ term for every in-between word, and those rarities are often large.

3 Reversibility and direction-dependent costs (5 pts)

Every DOGCAT move can be undone: if I can turn W_1 into W_2 by changing one letter, I can also turn W_2 back into W_1 . But being able to undo a move does not mean it costs the same going back. The cost is charged on the word I move into (the new letter for Scrabble, the word I land on for frequency), so the forward move and the backward move can have different costs.

3.1 Steps

Yes. Every move costs 1 no matter which way I go, so if the best path one way has some number of moves, walking it backwards gives a path the other way with the same number of moves.

3.2 Scrabble

No. The cost of a move is the value of the letter I put in, and that changes with direction. DOG and COG are one letter apart, so a single move is best both ways. Going DOG $\rightarrow$ COG I insert c, worth 3, but going COG $\rightarrow$ DOG I insert d, worth 2. So the best cost is 3.00 one way and 2.00 the other.

3.3 Frequency

No. A move costs $1 + \text{rarity}(w)$, where w is the word landed on, and the two directions land on different words. DOG $\rightarrow$ COG costs $1 + \text{rarity}(\text{cog}) = 12.39$, but COG $\rightarrow$ DOG costs $1 + \text{rarity}(\text{dog}) = 6.82$, because **cog** is the rarer word so it is more expensive to move toward. The same thing on a longer path in my runs: WHY to ASK came out to 38.86 one way and 37.76 the other.

4 LLM heuristic audit (10 pts)

4.1 Model and prompt

I used Claude. The exact prompt was:

In DOGCAT, the initial and goal states are lowercase words of the same length (three or four letters) and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. "More informative" means the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

4.2 Proposed heuristic

The model proposed charging the goal word's rarity for *every* position that still differs from the goal:

$$h_{\text{prop}}(n) = \sum_{i: n[i] \neq \text{goal}[i]} (1 + \text{rarity}(\text{goal})) = D(n) (1 + \text{rarity}(\text{goal})).$$

Its reasoning was that each wrong position needs at least one edit, and since a rare goal makes edits expensive, charging $1 + \text{rarity}(\text{goal})$ per wrong position should stay below the true cost while being larger than $h_0 = D(n)$.

4.3 Admissibility analysis

I did not take that reasoning at face value, so I checked whether the goal's rarity really applies to every remaining edit. It does not. Only the last edit is forced to enter the goal word. The earlier edits go through other words, and those words can be common, with much smaller rarity than the

goal. So multiplying $\text{rarity}(\text{goal})$ by $D(n)$ charges for rarity the path never actually pays, and the heuristic can come out too high.

4.4 Repair

I fixed the heuristic by adding the goal's rarity only once instead of once per wrong position. Only the final move is guaranteed to land on the goal word, so charging that rarity for every wrong position was the part that was too much:

$$h_{\text{fix}}(n) = \begin{cases} 0 & \text{if } D(n) = 0, \\ D(n) + \text{rarity}(\text{goal}) & \text{if } D(n) \geq 1. \end{cases}$$

This is the same frequency heuristic I used and justified in Question 1. It stays at or below the true cost (at least $D(n)$ moves remain, the last one enters the goal at cost $1 + \text{rarity}(\text{goal})$, and the rest cost at least 1 each), and it is still at least $h_0 = D(n)$, so it is more informative than the baseline.

4.5 Comparison vs. h_0

To see whether the repair was worth it, I ran A* on the two frequency problems from Question 2 twice each, once with plain Hamming h_0 and once with h_{fix} , reading the expanded count from `display=True`:

Problem	Heuristic	Cost	Nodes expanded
WHY-ASK (freq)	h_0 (Hamming)	38.86	755
WHY-ASK (freq)	h_{fix}	38.86	634
BEAR-DUCK (freq)	h_0 (Hamming)	38.24	720
BEAR-DUCK (freq)	h_{fix}	38.24	302

What I observed and what I chose. Both problems came back with the same optimal cost under either heuristic (38.86 and 38.24), and h_{fix} expanded fewer nodes than h_0 each time (634 vs 755, and 302 vs 720). So the repaired heuristic did the same job as Hamming but with less searching, which is why I went with it.

AI disclosure

I used ChatGPT while working on this assignment to help me understand the heuristics and with LaTeX formatting for this written section. For Question 4, I used Claude as the model. I verified everything myself: the code was checked by running `dcsolve.py` and `dctest.py` against the sample outputs in the handout.